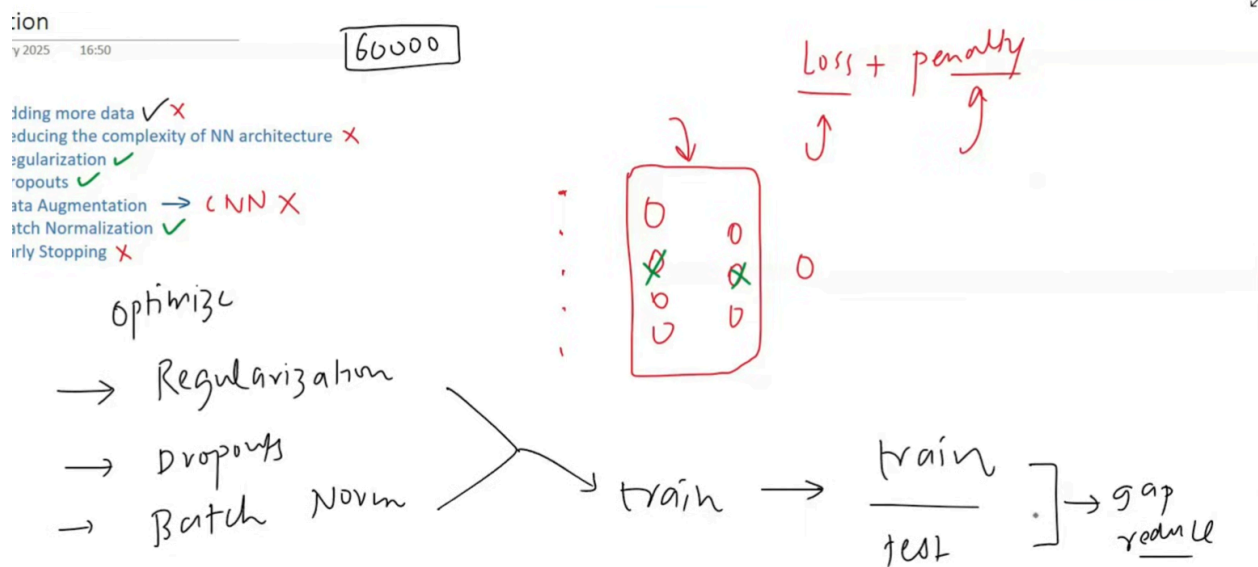


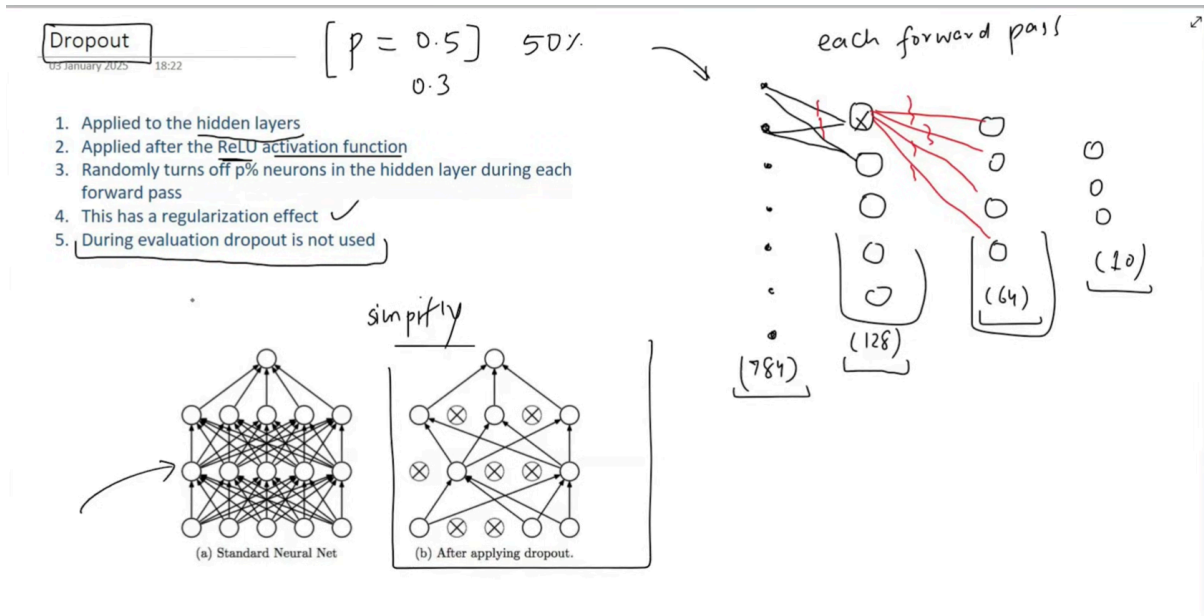
Optimizing : neural network performance

techniques to reduce the overfitting or model complexity :

since our training and test accuracy has a greater difference of around 10% we have these techniques to reduce the gap in our ann architecture from scratch.



▼ Dropouts



▼ Batch normalization

Here's a concise, less math-heavy breakdown of **Batch Normalization (BatchNorm)** in deep learning, structured for clarity and practical understanding:

1. Introduction: Why Normalize?

- **Goal:** Keep data well-scaled (mean ≈ 0 , variance ≈ 1) for stable and efficient training.
- **Problem with Unnormalized Data:**
 - Loss landscapes become stretched/asymmetrical.
 - Optimizers (e.g., gradient descent) take zigzag paths, slowing training.
 - Requires careful weight initialization and tiny learning rates to avoid instability.

2. Internal Covariate Shift

- **Issue:** In deep networks, even if input data is normalized, the distribution of activations changes layer-by-layer during training.

- **Impact:** Forces layers to constantly adapt to shifting input distributions, making training unstable.
- **BatchNorm Role:** Stabilizes activations by normalizing them within each mini-batch, ensuring consistent input distributions across layers.

Abstract

Training Deep Neural Networks is complicated by the fact that the distribution of each layer's inputs changes during training, as the parameters of the previous layers change. This slows down the training by requiring lower learning rates and careful parameter initialization, and makes it notoriously hard to train models with saturating nonlinearities. We refer to this phenomenon as *internal covariate shift*, and address the problem by normalizing layer inputs. Our method draws its strength from making nor-

down the training by requiring lower learning rates and careful parameter initialization, and makes it notoriously hard to train models with saturating nonlinearities. We refer to this phenomenon as *internal covariate shift*, and address the problem by normalizing layer inputs. Our method draws its strength from making normalization a part of the model architecture and performing the normalization *for each training mini-batch*. Batch Normalization allows us to use much higher learning rates and be less careful about initialization, and in some cases eliminates the need for Dropout. Applied to a state-of-the-art image classification model, Batch Nor-

3. How BatchNorm Works

A. During Training

1. For a mini-batch XXX with MMM examples:

- Compute mean (μ_B) and variance (σ_B^2) of the batch.

$$\mu_B$$

$$\sigma_B^2$$

- Normalize (ϵ prevents division by zero).

$$x = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

2. Scale & Shift:

- Apply learned parameters **gamma (γ)** and **beta (β)**:

$$y = \gamma \hat{x} + \beta$$

- Allows the network to undo normalization if needed (e.g., set $\gamma = \sigma$, $\beta = \mu$).

3. Bias Term Redundancy:

- BatchNorm cancels out the bias term in the layer, so it's often omitted.

4. Placement:

- Applied **before** the activation function (e.g., ReLU) to normalize pre-activations.

B. During Testing

- **No mini-batches:** Use stored moving averages of mean/variance (computed during training) for normalization.
- **Formula:** Exponential moving average (EMA) updates:

$$\mu_{EMA} = \alpha \mu_{EMA} + (1 - \alpha) \mu_B$$
(α = decay factor, e.g., 0.9).

$$\mu_{EMA} = \alpha \mu_{EMA} + (1 - \alpha) \mu_B \quad \sigma_{EMA} = \alpha \sigma_{EMA} + (1 - \alpha) \sigma_B$$

4. Why BatchNorm Helps (Beyond the Original Explanation)

- **Smooths the Loss Landscape:** Reduces jaggedness, so small weight changes don't cause loss spikes.
- **Enables Higher Learning Rates:** Speeds up training.
- **Acts as a Regularizer:** Noise from batch statistics reduces overfitting.

Key Equations (Simplified)

Step	Equation
Normalization	$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$
Scale & Shift	$y = \gamma \hat{x} + \beta$
Exponential Moving Average	$\mu_{EMA} = \alpha \mu_{EMA} + (1 - \alpha) \mu_B$

Practical Takeaways

- **Where to Use:** After linear/conv layers, before activations.
- **Parameters:** γ and β are learned during training.
- **Testing:** Use stored EMA statistics, not batch stats.

Want to dive deeper? Check out the [original BatchNorm paper](#) and "[How Does Batch Normalization Help Optimization?](#)" for advanced insights.

▼ Regularization

L1 (Lasso) and L2 (Ridge) regularization are **techniques used to prevent machine learning models from overfitting by adding a penalty term to the loss function**. L1 adds the sum of absolute weight values, driving some coefficients to zero for feature selection, while L2 adds the sum of squared weights, shrinking them to create smaller, non-zero values.

Key Differences and Details:

- **L1 Regularization (Lasso):**
 - **Mechanism:** Adds $\lambda \sum_{j=1}^p |\beta_j|$ to the loss function.
 - **Output:** Produces sparse models, meaning it drives less important feature weights to exactly zero.
 - **Use Case:** Ideal for high-dimensional data when you want automatic feature selection (few features are actually important).
 - calls this "Lasso" or "Least Absolute Shrinkage and Selection Operator".
- **L2 Regularization (Ridge):**
 - **Mechanism:** Adds $\lambda \sum_{j=1}^p \beta_j^2$ to the loss function.
 - **Output:** Shrinks weights to be small, but rarely zero. It spreads the penalty among all features, leading to more stable, non-sparse models.
 - **Use Case:** Good for reducing overall model complexity when you have multicollinearity (features are highly correlated).

- **Towards Data Science** mentions L2 is also known as "Ridge" regression.
- **Common Goals:** Both methods aim to minimize the loss function while limiting the magnitude of model coefficients (β), resulting in simpler, more generalized models that perform better on new data.
 - emphasizes that "L2 regularization works smoothly while using standard gradient descent algorithms as it is always differentiable."
- **Hyperparameter :** A tuning parameter (λ or α) controls the strength of the penalty; higher values increase regularization, leading to simpler models but higher bias.

Both techniques are crucial for preventing models from memorizing the training data, ultimately reducing overfitting.

When to Use L2 Regularization (Weight Decay):

- **General Performance:** Use as the default regularization method when you want to reduce overfitting but do not need feature selection.
- **Correlated Features:** Use when features are highly correlated (multicollinearity), as L2 distributes the weight penalty among them, enhancing model stability.
- **Small/Moderate Complexity:** Use to improve accuracy by penalizing the square of weights, ensuring no single feature dominates the model's output. [3, 4, 8, 9, 10, 11]

Key Differences and Tips:

- **Optimization:** L2 is generally preferred because it works better with gradient descent (smoother).
- **Combination:** You can use both L1 and L2 simultaneously (Elastic Net) for a balance of feature selection and stability.
- **Implementation:** In frameworks like TensorFlow/PyTorch, L2 is often applied using in the optimizer. [3, 5, 8, 12, 13]

In summary, choose **L1** for sparsity/selection and **L2** for general stabilization and performance enhancement.
